Migración a persistencia en **PostgreSQL**

Objetivo

Migrar el almacenamiento de blueprints desde una implementación en memoria hacia una base de datos PostgreSQL, manteniendo el contrato definido por la interfaz BlueprintPersistence.

Configuración de PostgreSQL

1. **Levantamiento Docker**

Primero levantamos el servidor Docker con el siguiente comando:

docker run --name blueprints-postgres -e POSTGRES_DB=blueprintsdb -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres:16

Y validamos con **docker ps**

2. **Creación de la estructura de base de datos**

Primero nos conectamos al psql con el siguiente comando

docker exec -it blueprints-postgres psql -U postgres -d blueprintsdb

Dentro del dicho comando se crean las siguientes tablas

```
CREATE TABLE blueprints (
  author VARCHAR(100) NOT NULL,
  name VARCHAR(100) NOT NULL,
  PRIMARY KEY (author, name)
);


CREATE TABLE points (
  id SERIAL PRIMARY KEY,
  author VARCHAR(100) NOT NULL,
  blueprint_name VARCHAR(100) NOT NULL,
  x INTEGER NOT NULL,
```

```
    y INTEGER NOT NULL,
    FOREIGN KEY (author, blueprint_name)
        REFERENCES blueprints(author, name)
        ON DELETE CASCADE
);
```

Y salimos con \q

Luego agregamos las dependencias

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

Y configuramos el **application.properties** del postman

```
spring.mvc.pathmatch.matching-strategy=ant_path_matcher
spring.datasource.url=jdbc:postgresql://localhost:5432/blueprint
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.sql.init.mode=never
```

Se utilizó Docker para desplegar una instancia de PostgreSQL 16.

Se creó la base de datos blueprintsdb y las tablas:

- blueprints
- points

Se definió una clave primaria compuesta (author, name) y una relación uno a muchos entre blueprint y puntos.

Comandos útiles

Detener contenedor

docker stop blueprints-postgres

Volverlo a iniciar

docker start blueprints-postgres

Eliminar contenedor

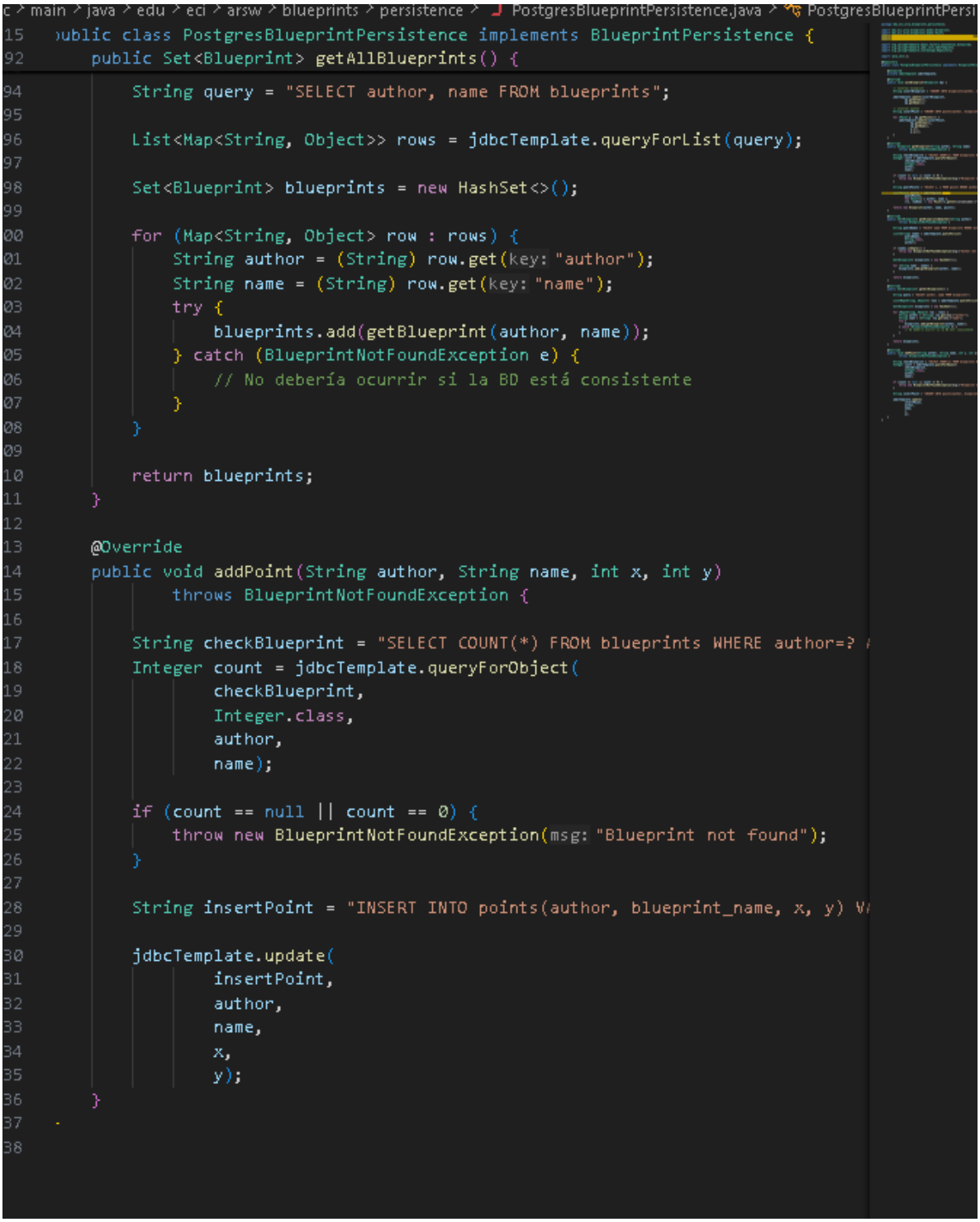
docker rm -f blueprints-postgres

3. **Implementación PostgresBlueprintPersistence**

Guarda y consulta los Blueprint en una base de datos PostgreSQL usando JdbcTemplate.

```java
@Repository
public class PostgresBlueprintPersistence implements BlueprintPersistence {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Override
    public void saveBlueprint(Blueprint bp) {

        // Insertar blueprint
        String insertBlueprint = "INSERT INTO blueprints(author, name) VALUES (?,

        jdbcTemplate.update(insertBlueprint,
                bp.getAuthor(),
                bp.getName());

        // Insertar puntos
        String insertPoint = "INSERT INTO points(author, blueprint_name, x, y) V

        for (Point p : bp.getPoints()) {
            jdbcTemplate.update(insertPoint,
                    bp.getAuthor(),
                    bp.getName(),
                    p.x(),
                    p.y());
        }
    }

    @Override
    public Blueprint getBlueprint(String author, String name)
            throws BlueprintNotFoundException {

        String checkBlueprint = "SELECT COUNT(*) FROM blueprints WHERE author=? A
        Integer count = jdbcTemplate.queryForObject(
                checkBlueprint,
                Integer.class,
                author,
                name);

        if (count == null || count == 0) {
            throw new BlueprintNotFoundException(msg: "Blueprint not found");
        }

        String queryPoints = "SELECT x, y FROM points WHERE author=? AND bluepri
```

```
        String queryPoints = "SELECT x, y FROM points WHERE author=? AND bluep

        List<Point> points = jdbcTemplate.query(
                queryPoints,
                new Object[] { author, name },
                (rs, rowNum) -> new Point(rs.getInt(columnLabel: "x"), rs.getIr

        return new Blueprint(author, name, points);
    }

    @Override
    public Set<Blueprint> getBlueprintsByAuthor(String author)
            throws BlueprintNotFoundException {

        String queryNames = "SELECT name FROM blueprints WHERE author=?";

        List<String> names = jdbcTemplate.queryForList(
                queryNames,
                String.class,
                author);

        if (names.isEmpty()) {
            throw new BlueprintNotFoundException(msg: "Author not found");
        }

        Set<Blueprint> blueprints = new HashSet<>();

        for (String name : names) {
            blueprints.add(getBlueprint(author, name));
        }

        return blueprints;
    }

    @Override
    public Set<Blueprint> getAllBlueprints() {

        String query = "SELECT author, name FROM blueprints";

        List<Map<String, Object>> rows = jdbcTemplate.queryForList(query);

        Set<Blueprint> blueprints = new HashSet<>();

        for (Map<String, Object> row : rows) {
            String author = (String) row.get(key: "author");
            String name = (String) row.get(key: "name");
            try {
                blueprints.add(getBlueprint(author, name));
```


c > main > java > edu > eci > arsw > blueprints > persistence > PostgresBlueprintPersistence.java > PostgresBlueprintPersi
15 ublic class PostgresBlueprintPersistence implements BlueprintPersistence {
92 public Set<Blueprint> getAllBlueprints() {
94 String query = "SELECT author, name FROM blueprints";
96 List<Map<String, Object>> rows = jdbcTemplate.queryForList(query);
98 Set<Blueprint> blueprints = new HashSet<>();
00 for (Map<String, Object> row : rows) {
01 String author = (String) row.get(key: "author");
02 String name = (String) row.get(key: "name");
03 try {
04 blueprints.add(getBlueprint(author, name));
05 } catch (BlueprintNotFoundException e) {
06 // No debería ocurrir si la BD está consistente
07 }
08 }
09
10 return blueprints;
11 }
12
13 @Override
14 public void addPoint(String author, String name, int x, int y)
15 throws BlueprintNotFoundException {
16
17 String checkBlueprint = "SELECT COUNT(*) FROM blueprints WHERE author=? A
18 Integer count = jdbcTemplate.queryForObject(
19 checkBlueprint,
20 Integer.class,
21 author,
22 name);
23
24 if (count == null || count == 0) {
25 throw new BlueprintNotFoundException(msg: "Blueprint not found");
26 }
27
28 String insertPoint = "INSERT INTO points(author, blueprint_name, x, y) V
29
30 jdbcTemplate.update(
31 insertPoint,
32 author,
33 name,
34 x,
35 y);
36 }
37
38